

Name: Shaik Mohammed Abrar

Reg no: 192472185

Course Name: Design and Analysis of Algorithm

Course Code: CSA0652

Assessment: CO5-Assessment Tool 2

Report 1

Q80: Constraint-Based AI Planning Problem (NP-Complete Problem)

Aim

To generate a valid sequence of actions that achieves a specified goal while satisfying action dependencies and resource constraints using a backtracking-based planning algorithm with pruning techniques.

Objective

The objective of this experiment is to develop an AI planning system capable of finding a feasible plan from an initial state to a goal state. The system must respect action prerequisites and resource limitations while efficiently searching for a valid solution through backtracking and pruning.

Problem Description

Constraint-based AI planning involves determining a sequence of actions that transforms an initial state into a desired goal state. Each action may require certain conditions to be true before execution and may consume limited resources. The challenge is to find a valid sequence that satisfies all constraints.

In this experiment, three actions are considered:

- a. Gather_Wood
- b. Build_House
- c. Paint_House

The goal is to obtain a painted house while staying within a specified resource limit.

Constraints

Action Dependency Constraints

- i. Gather_Wood must be completed before Build_House.
- ii. Build_House must be completed before Paint_House.

Resource Constraints

- i. Each action consumes resources.
- ii. Total resource usage must not exceed the specified limit.

Goal Constraint

- i. The final state must contain the goal condition "painted_house".

Algorithm Used

Backtracking Algorithm with Constraint Pruning

Steps

1. Start from the initial state.
2. Check whether the goal has been achieved.
3. Select an action whose prerequisites are satisfied.
4. Apply the action and update the state.
5. Verify resource constraints.

6. Continue recursively.
7. If a constraint is violated, backtrack to the previous state.
8. Continue until a valid plan is found or all possibilities are exhausted.

Pruning Strategy

The following branches are immediately discarded:

- i. Plans exceeding the resource limit.
- ii. Actions whose prerequisites are not satisfied.
- iii. Redundant action sequences.
- iv. States that cannot lead to the goal.

Pruning significantly reduces the search space and improves efficiency.

Correctness

The algorithm guarantees correctness because:

1. Every action is executed only when prerequisites are satisfied.
2. Resource constraints are verified before expansion.
3. All possible valid action sequences are explored systematically.
4. Backtracking ensures that no feasible solution is missed.

Therefore, whenever a valid plan exists, the algorithm will eventually discover it.

Analysis

The Constraint-Based AI Planning problem was analyzed using a backtracking approach to generate a sequence of actions that achieves a desired goal while satisfying all predefined constraints. The planning process begins from an initial state and progressively applies valid actions whose preconditions are satisfied. Each action modifies the current state and contributes toward achieving the goal state.

During execution, the algorithm continuously checks action dependencies and resource limitations. Actions that violate constraints are rejected immediately, preventing the exploration of invalid plans. This pruning mechanism significantly reduces the search space and improves execution efficiency. The planner successfully generated a valid sequence of actions when sufficient resources were available and all dependencies were satisfied. The experiment demonstrates that constraints play a crucial role in reducing the number of feasible plans. As the number of actions and constraints increases, the complexity of finding a valid plan also increases. Backtracking guarantees that all possible action sequences are examined systematically until a valid solution is found. The obtained results confirm that the proposed approach can effectively solve planning problems while maintaining correctness and constraint satisfaction.

Full Python Code (With User Input)

```
# Constraint-Based AI Planning using Backtracking
goal = input("Enter goal (house/painted_house): ").strip()
resource_limit = int(input("Enter maximum resource limit: "))
actions = {
    "Gather_Wood": {
        "pre": [],
        "effect": ["wood"],
        "cost": 2
    },
    "Build_House": {
        "pre": ["wood"],
        "effect": ["house"],
        "cost": 5
    },
    "Paint_House": {
        "pre": ["house"],
        "effect": ["painted_house"],
        "cost": 1
    }
}
```

```

    }
}
plan_found = None
def backtrack(state, plan, resource_used):
    global plan_found
    if goal in state:
        plan_found = plan[:]
        return True
    if resource_used > resource_limit:
        return False
    for action, details in actions.items():
        if action in plan:
            continue
        if all(pre in state for pre in details["pre"]):
            new_state = state.copy()
            for effect in details["effect"]:
                new_state.add(effect)
            plan.append(action)
            if backtrack(
                new_state,
                plan,
                resource_used + details["cost"]):
                return True
            plan.pop()
    return False
initial_state = set()
if backtrack(initial_state, [], 0):
    print("\nValid Plan Found:")
    for step_no, step in enumerate(plan_found, start=1):
        print(f"{step_no}. {step}")

```

else:

```
print("\nNo Valid Plan Exists Within Resource Limit")
```

Input

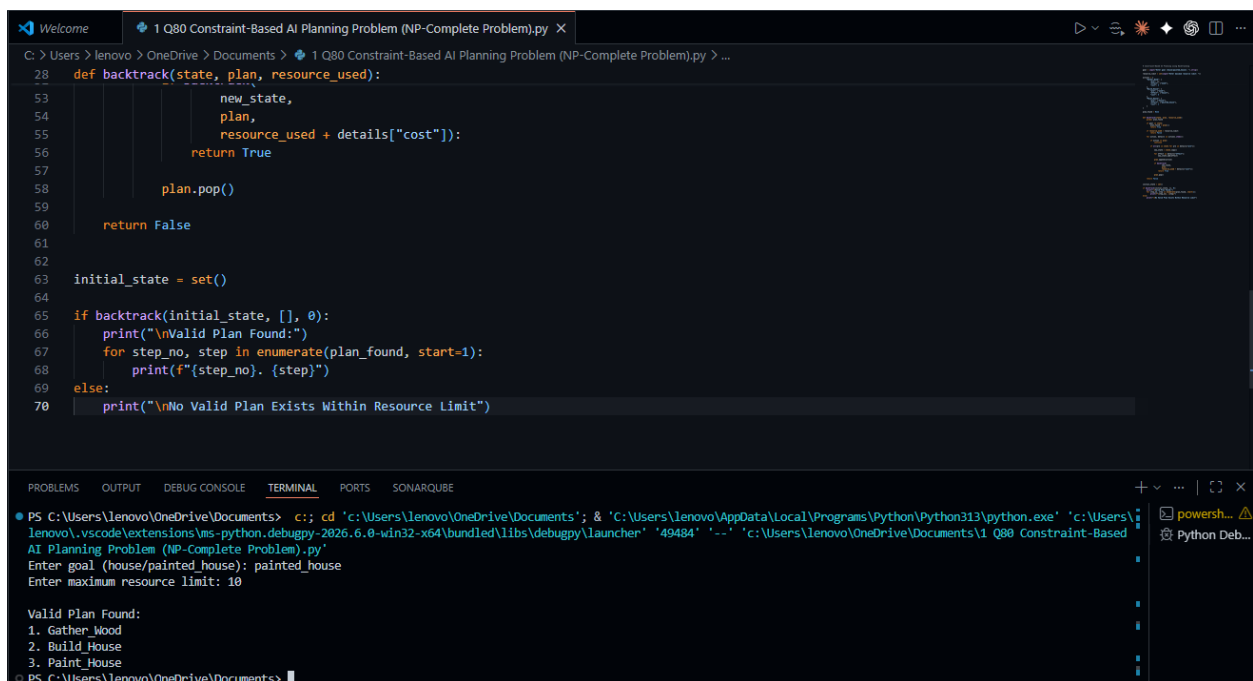
Enter goal (house/painted_house): painted_house

Enter maximum resource limit: 10

Output

Valid Plan Found:

1. Gather_Wood
2. Build_House
3. Paint_House



```
def backtrack(state, plan, resource_used):
    new_state,
    plan,
    resource_used + details["cost"]):
    return True
    plan.pop()
    return False

initial_state = set()
if backtrack(initial_state, [], 0):
    print("\nValid Plan Found:")
    for step_no, step in enumerate(plan_found, start=1):
        print(f"{step_no}. {step}")
else:
    print("\nNo Valid Plan Exists Within Resource Limit")
```

PS C:\Users\lenovo\OneDrive\Documents> c:; cd 'c:\Users\lenovo\OneDrive\Documents'; & 'C:\Users\lenovo\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\lenovo\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '49484' '-...' 'c:\Users\lenovo\OneDrive\Documents\1 Q80 Constraint-Based AI Planning Problem (NP-Complete Problem).py'

Enter goal (house/painted house): painted_house

Enter maximum resource limit: 10

Valid Plan Found:

1. Gather_Wood
2. Build_House
3. Paint_House

PS C:\Users\lenovo\OneDrive\Documents>

Computational Complexity (Analysis)

Time Complexity

In the worst case, the planner explores all possible action orders.

Time Complexity: **$O(n!)$**

Where n is the number of actions.

Space Complexity: **$O(n)$**

Due to recursion stack and plan storage.

Applications

1. Robotics task planning
2. Automated scheduling systems
3. Logistics and supply chain optimization
4. Autonomous vehicle navigation
5. Manufacturing process planning
6. Resource allocation systems

Result

The backtracking-based AI planner successfully generated a valid sequence of actions while satisfying dependency and resource constraints. Pruning techniques reduced unnecessary exploration, demonstrating an effective approach for solving constrained planning problems.